The relation matrix inside `ic.pipeline`: sparse-cado against dense full-rank, |F| = 1,024 to 16,384

2026-10-19 · study `research/ic_linalg_pipeline_20261019` · protocol frozen in `a75e9b407` before data · sessions `ECBS1hf4d23305c958` (A, N ≤ 8,192) and `ECBS1h0c4c69816435` (B, N = 16,384) · curve `icv1-fp32-t7319-90da43e9` ($p = 2^{32} - 5$, prime order r = 4294959973) · reference `rho.negation`.

Question

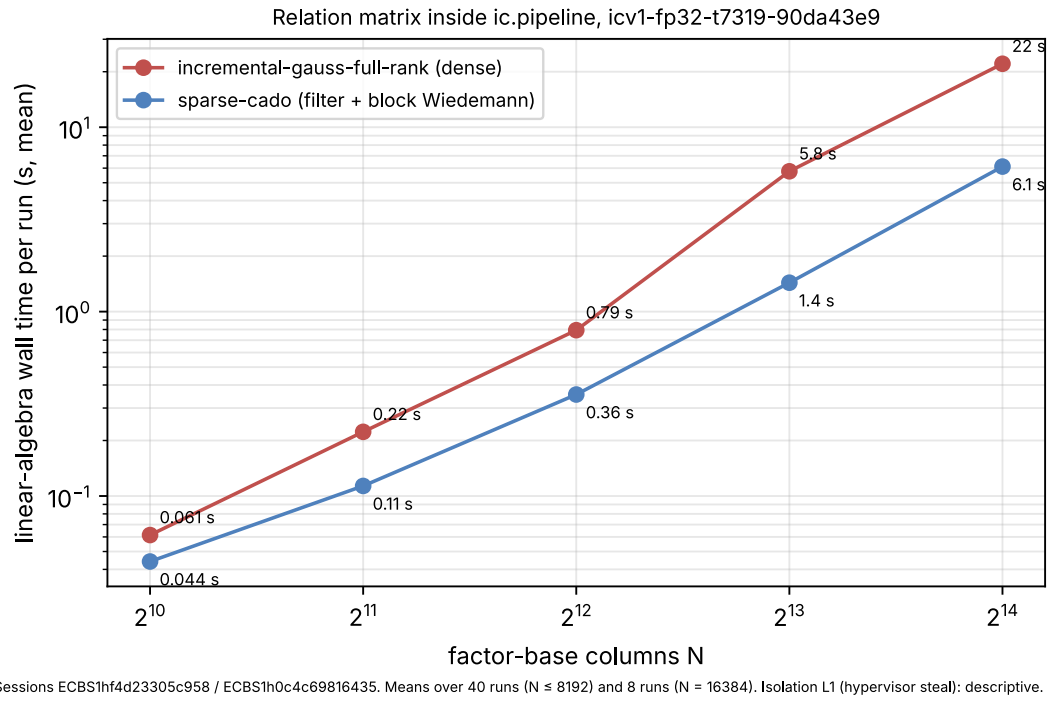
What does choosing `linalg=sparse-cado` (CADO-style filter, then block Wiedemann) over `linalg=incremental-gauss-full-rank` change in a full ecbench `ic.pipeline` run as the factor base grows? Both use the full-rank stop rule, with the same factor base (`prime-abscissa:size=N`), the same oracle (`mitm:negation_folded=1`) and the same relation stream. Session A ran 8 targets × 5 rounds (+1 warm-up); session B ran 4 targets × 2 rounds (+1 warm-up). All 516 records verified, and the audits are OK (24 and 6 exact replays).

Result

S does not change. The ecbench `ops` ratio cado/dense is 1.0000, interval [1, 1], at every N. Linear algebra is unpriced in `ic.pipeline`. Priced across the §5a band (w = 0, 0.1 and 1 GAE per unpriced operation, every arm alike), the two tie at w = 0, and dense is lower by at most 0.27% at w > 0. The two are not ranked.

N	row_ops dense	row_ops cado	cado/dense	LA wall dense	LA wall cado	cado/dense wall, Σ (pair range)
1,024	28,749	43,991	1.53	61 ms	44 ms	0.72 (0.59–0.95)
2,048	61,209	92,929	1.52	223 ms	113 ms	0.51 (0.41–0.63)
4,096	129,316	194,343	1.50	793 ms	355 ms	0.45 (0.39–0.57)
8,192	281,907	422,323	1.50	5.77 s	1.44 s	0.25 (0.21–0.32)
16,384	613,491	918,135	1.50	22.1 s	6.12 s	0.28 (0.23–0.32)

Means per run; ratios are Σ over (workload, round) pairs, 40 per N (8 at 16,384). The A/A control (`cado-4096-aa`) has identical row_ops and an LA wall ratio of 1.02 (pair range 0.83–1.31). **Every wall figure is descriptive**: the IC runs earned L1 (41–47 ticks of hypervisor steal), and ecbench admitted no wall pair.



Sessions ECBS1hf4d23305c958 / ECBS1h0c4c69816435. Means over 40 runs (N ≤ 8192) and 8 runs (N = 16384). Isolation L1 (hypervisor steal): descriptive.

Figure 1: Linear-algebra wall time per run against N (means; descriptive, L1).

sparse-cado does about 1.5× more counted work. Two-term relations barely fill in, so the dense elimination's counted work stays near-linear. But sparse-cado spends 2–4× less linear-algebra wall time from N = 2,048 up: the dense method's column scans are not counted in `row_ops`. From 2,048 up, the per-pair ranges lie below the A/A range.

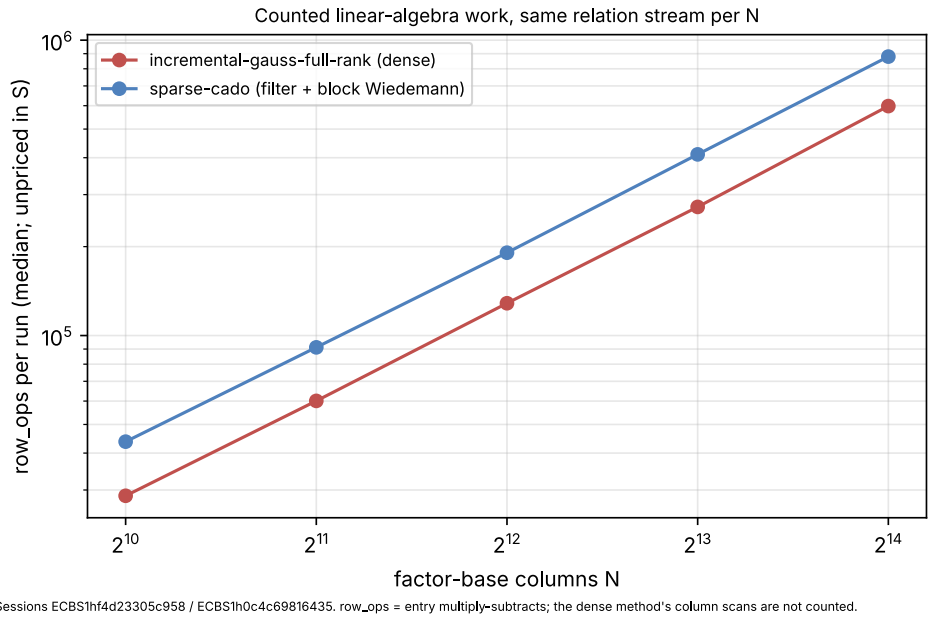


Figure 2: Counted linear-algebra work (row_ops, median per run).

End to end the matrix is a small share. The oracle's $|F|^2$ pair-table set-up takes 84% of a dense run's wall time at 8,192 and 89% at 16,384, so the whole-run wall median ratio cado/dense is only 0.89 and 0.90 (descriptive).

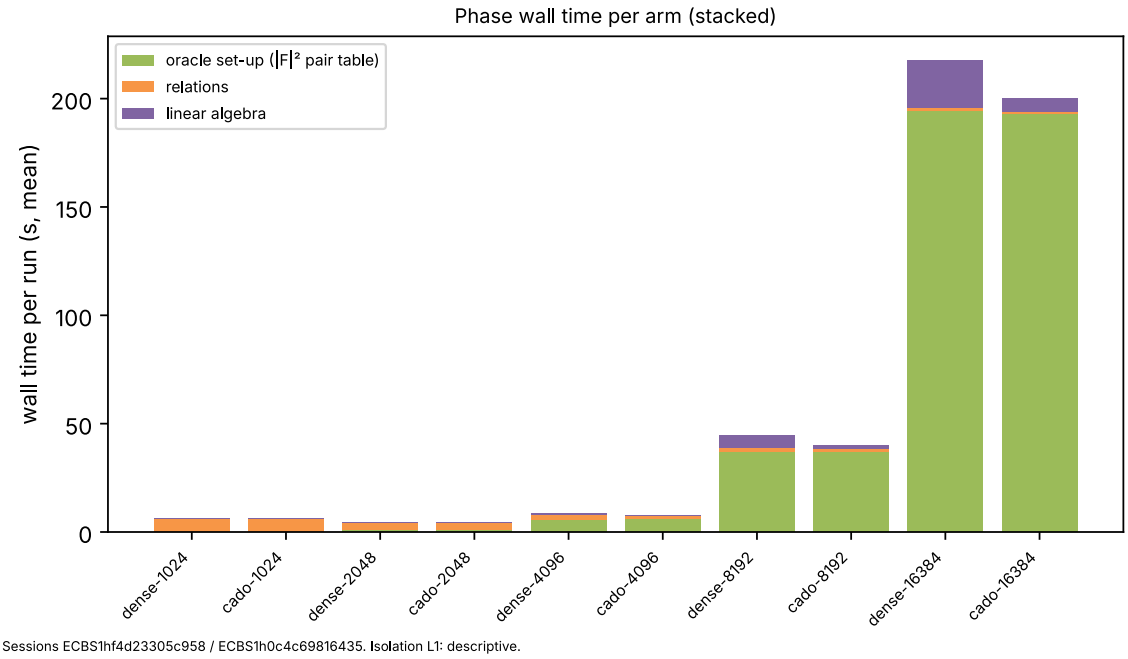


Figure 3: Phase wall time per arm (means, stacked; descriptive).

Distance remaining to rho

`ops` ratio S_{cado} / S_{rho} (w = 0, cluster 95% intervals); dense's is identical.

N	1,024	2,048	4,096	8,192	16,384
S_{IC} / S_{rho}	156 [124, 203]	149 [119, 190]	335 [270, 428]	1,201 [970, 1,542]	4,054 [2,649, 7,479]

The best size is 2,048; beyond it the $|F|^2$ set-up outgrows the cheaper relation collection. At w = 1 the gaps are 150–2,052×.

Classification

Engineering, not an advance (AGENTS.md §3). The matrix does not change S here. sparse-cado cuts its wall time (descriptively) at the cost of more counted multiply-subtracts, and `row_ops` is the wrong proxy for the dense method's cost. Neither touches the bottleneck: the oracle set-up, and a gap of at least 149× to rho. Predictions P1–P4 held (P3

descriptively). Deviations: the replay counts were not fixed by the protocol (24/480 and 6/36, all exact), and the two sessions record different environment class ids on the same host and binary.